

MMML cli_water Tutorial (Typst Draft)

This document is a practical walkthrough of the scripts in `cli_water/`. It is meant as a living tutorial that you can expand while you run the pipeline.

What this tutorial covers

- The script-driven workflow from structure setup to model retraining.
- Data flow between each numbered script.
- Key outputs at each stage.
- Optional/advanced branches (XX_*, active learning, retraining).

Directory map (`cli_water/`)

- `01_make_res_cli.sh`: create residue/topology and initial structure.
- `02_make_box_cli.sh`: build a periodic box (PackMol).
- `04_pyscf_dft_cli_full.sh`: DFT (energy, gradient, hessian, harmonic, thermo).
- `06_normal_mode_sample_cli.sh`: normal-mode sampling at several amplitudes.
- `07_pyscf_evaluate_cli.sh`: evaluate sampled structures with PySCF (`--esp`).
- `08_fix_and_split_cli.sh`: unit fixes + train/valid/test splits.
- `09_physnet_train_cli.sh`: PhysNet training.
- `10_physnet_dcmnet_train_cli.sh`: joint PhysNet + DCMNet training.
- `11_run_pycharmm_cli.sh`: CHARMM heat/equilibration baseline run.
- `12_charmm_esp_dipole_comparison.sh`: compare CHARMM and ML dipole/ESP behavior.
- `13_physnet_md.sh`: generate PhysNet MD trajectory.
- `14_active_learning.sh`: extract useful MD frames for re-labeling.
- `15_physnet_retrain_cli.sh`: retrain using one extended split.
- `15.1_physnet_retrain_cli.sh`: retrain using multiple extended splits.
- `run_full_training.sh`: convenience script for a full end-to-end run.
- `shared.source`: shared defaults (trainer path, checkpoints, dataset paths).
- `splits_extended/` and `splits_extended2/`: additional split datasets for retraining.

Recommended run order

This is the high-level order encoded by script dependencies:

1. `01_make_res_cli.sh`
2. `02_make_box_cli.sh`
3. `04_pyscf_dft_cli_full.sh`
4. `06_normal_mode_sample_cli.sh`
5. `07_pyscf_evaluate_cli.sh`
6. `08_fix_and_split_cli.sh`
7. `09_physnet_train_cli.sh`
8. `10_physnet_dcmnet_train_cli.sh`
9. `13_physnet_md.sh`
10. `14_active_learning.sh`
11. `15_physnet_retrain_cli.sh` or `15.1_physnet_retrain_cli.sh`

`11_run_pycharmm_cli.sh` and `12_charmm_esp_dipole_comparison.sh` are side branches for MM baseline and diagnostics.

Step-by-step notes

Step 01: Make residue

Purpose: generate residue-level structure and topology files.

```
mmml make-res --res TIP3 --skip-energy-show
```

Main outputs:

- `pdb/initial.pdb`
- `psf/initial.psf`

- xyz/initial.xyz

Notes:

- Script comments mention CYBZ in places, but the command uses TIP3.
- Requires CHARMM/PyCHARMM environment.

Step 02: Build box

Purpose: create a packed periodic system from the residue.

```
mmml make-box --res CYBZ --n 2 --side_length 25.0
```

Main output:

- pdb/init-packmol.pdb

Notes:

- Depends on step 01 output.
- Requires PackMol installed.

Step 04: Full DFT with PySCF

Purpose: create high-quality QM reference data and harmonic information.

```
mmml pyscf-dft --mol xyz/initial.xyz \
  --energy --gradient --hessian --harmonic --thermo \
  --output out/04_results
```

Main outputs:

- out/04_results.npz
- out/04_results.h5

Notes:

- Hessian/harmonic/thermo can be expensive.
- out/04_results.h5 is required by normal mode sampling.

Step 06: Normal mode sampling

Purpose: generate perturbed geometries for dataset construction.

The script samples multiple amplitudes (0.2 to 0.5) and writes:

```
mmml normal-mode-sample -i out/04_results.h5 -o out/06_sampled2.npz --amplitude 0.2 --max-samples 200
mmml normal-mode-sample -i out/04_results.h5 -o out/06_sampled3.npz --amplitude 0.3 --max-samples 200
mmml normal-mode-sample -i out/04_results.h5 -o out/06_sampled4.npz --amplitude 0.4 --max-samples 200
mmml normal-mode-sample -i out/04_results.h5 -o out/06_sampled5.npz --amplitude 0.5 --max-samples 200
```

Typical contents:

- R, Z, N

Step 07: Evaluate sampled geometries

Purpose: label sampled structures with QM properties (including ESP grids).

```
mmml pyscf-evaluate -i out/06_sampled2.npz -o out/07_evaluated2.npz --esp
mmml pyscf-evaluate -i out/06_sampled3.npz -o out/07_evaluated3.npz --esp
mmml pyscf-evaluate -i out/06_sampled4.npz -o out/07_evaluated4.npz --esp
mmml pyscf-evaluate -i out/06_sampled5.npz -o out/07_evaluated5.npz --esp
```

Typical contents:

- R, Z, N, E, F, Dxyz, esp, esp_grid

Step 08: Fix units and split datasets

Purpose: convert to training units and build deterministic splits.

```
mmml fix-and-split --efd out/07_evaluated.npz --output-dir out/splits --atomic-ref pbe0/def2-tzvp
```

Main outputs:

- out/splits/energies_forces_dipoles_{train,valid,test}.npz
- out/splits/grids_esp_{train,valid,test}.npz

Based on splits_extended*/README.md, expected conversions include:

- energy: Hartree -> eV
- force: Hartree/Bohr -> eV/Angstrom
- dipole: Debye -> e*Angstrom
- ESP grid coordinates converted to physical Angstrom positions

Step 09: Train PhysNet

Purpose: train baseline PhysNet on E/F/D data.

The script loads defaults from shared.source:

```
python "$PHYSNET_TRAINER" \  
  --train "$PHYSNET_TRAIN_NPZ" \  
  --valid "$PHYSNET_VALID_NPZ" \  
  --natoms "$PHYSNET_NATOMS" \  
  --epochs "$PHYSNET_EPOCHS" \  
  --batch-size "$PHYSNET_BATCH" \  
  --charges \  
  --zbl \  
  --name "$PHYSNET_NAME" \  
  --ckpt-dir "$PHYSNET_CKPT_DIR"
```

Main output:

- "\$PHYSNET_CKPT_DIR/\$PHYSNET_NAME/" (run directory)

Step 10: Joint PhysNet + DCMNet training

Purpose: co-train on E/F/D and ESP grid data.

```
python -m mmml.cli.misc.train_joint \  
  --train-efd out/splits/energies_forces_dipoles_train.npz \  
  --train-esp out/splits/grids_esp_train.npz \  
  --valid-efd out/splits/energies_forces_dipoles_valid.npz \  
  --valid-esp out/splits/grids_esp_valid.npz \  
  --epochs 1000 \  
  --batch-size 1 \  
  --name eg_joint \  
  --ckpt-dir ~/ckpts --plot-results --plot-freq 0
```

Notes:

- run_full_training.sh can optionally pass --physnet-checkpoint if a matching PhysNet run exists.

Step 11: Pure CHARMM equilibration

Purpose: run MM-only baseline heating/equilibration.

```
uv run mmml run-pycharmm --pdbfile pdb/init-packmol.pdb --cell 25.0
```

Main outputs:

- pdb/heat.pdb, pdb/equi.pdb
- dcd/heat.dcd, dcd/equi.dcd
- res/heat.res, res/equi.res

Step 12: CHARMM vs ML comparison

Purpose: compare behavior on a test split.

```
python -m mmml.cli.misc.compare_charmm_ml \  
  --checkpoint ~/ckpts/eg_joint \  
  --valid-efd out/splits/energies_forces_dipoles_test.npz \  
  --valid-esp out/splits/grids_esp_test.npz
```

```
--valid-esp out/splits/grids_esp_test.npz \
--pdb pdb/initial.pdb \
--n-samples 50 \
--out-dir charmm_ml_comparison
```

Step 13: PhysNet MD sampling

Purpose: run MD to create new candidate structures.

```
mmmml physnet-md \
--checkpoint "$PHYSNET_CHECKPOINT" \
--data "$PHYSNET_TRAIN_NPZ" \
-o "$PHYSNET_MD_OUT"
```

Main outputs:

- physnet_ase.traj
- physnet_ase_final.xyz
- physnet_jaxmd.xyz

Step 14: Active learning extraction

Purpose: filter MD frames (e.g., by temperature) for expensive QM relabeling.

```
mmmml active-learning \
-i "$PHYSNET_MD_TRAJ" \
-o "$ACTIVE_LEARNING_OUT" \
--max-temp 300
```

Follow-up path suggested by the script:

```
mmmml pyscf-evaluate -i out/active_learning.npz -o out/md_evaluated.npz --esp
mmmml fix-and-split --efd out/splits/energies_forces_dipoles_train.npz out/md_evaluated.npz -o
out/splits_extended
```

Step 15 / 15.1: Retraining on extended data

Purpose: continue training from an earlier checkpoint with additional active-learning data.

Single extension:

```
python "$PHYSNET_TRAINER" \
--train out/splits/energies_forces_dipoles_train.npz splits_extended/
energies_forces_dipoles_train.npz \
--valid out/splits/energies_forces_dipoles_valid.npz splits_extended/
energies_forces_dipoles_test.npz \
--natoms "$PHYSNET_NATOMS" \
--epochs "$PHYSNET_EPOCHS" \
--name "$PHYSNET_NAME" \
--ckpt-dir "$PHYSNET_CKPT_DIR" \
--restart "$PHYSNET_CHECKPOINT"
```

Multiple extensions:

```
python "$PHYSNET_TRAINER" \
--train out/splits/energies_forces_dipoles_train.npz splits_extended/
energies_forces_dipoles_train.npz splits_extended2/energies_forces_dipoles_train.npz \
--valid out/splits/energies_forces_dipoles_valid.npz splits_extended/
energies_forces_dipoles_valid.npz splits_extended2/energies_forces_dipoles_valid.npz \
--natoms "$PHYSNET_NATOMS" \
--batch-size "$PHYSNET_BATCH" \
--epochs 1500 \
--charges \
--name "$PHYSNET_NAME" \
--ckpt-dir "$PHYSNET_CKPT_DIR" \
--restart "$PHYSNET_CHECKPOINT"
```

Optional scripts (XX_*)

- `XX_pyscf_dft_cli.sh`: single-geometry DFT energy example on water.
- `XX_pyscf_mp2_cli.sh`: single-geometry MP2 energy example on water.

These are useful as sanity checks before launching the full dataset workflow.

shared.source variables you will likely edit

- `PHYSNET_TRAINER`
- `PHYSNET_CKPT_DIR`
- `PHYSNET_NAME`
- `PHYSNET_CHECKPOINT`
- `PHYSNET_NATOMS`
- `PHYSNET_EPOCHS`
- `PHYSNET_BATCH`
- `PHYSNET_TRAIN_NPZ`
- `PHYSNET_VALID_NPZ`
- `PHYSNET_MD_OUT`
- `PHYSNET_MD_TRAJ`
- `ACTIVE_LEARNING_OUT`

Fast start options

Option A: full pipeline script

```
bash cli_water/run_full_training.sh
```

Option B: manual progression

Run each script in order to inspect intermediate outputs and troubleshoot early.

Known mismatches to watch

- Some comments still reference `examples/mml_tutorial/cli`, while this copy is under `cli_water/`.
- Step `01_make_res_cli.sh` echoes CYBZ in a message but executes `--res TIP3`.
- Step `08_fix_and_split_cli.sh` uses `out/07_evaluated.npz`, but `07_pyscf_evaluate_cli.sh` currently writes numbered files (`07_evaluated2..5.npz`).

These are not necessarily wrong, but they are common sources of confusion when reproducing the workflow.

Next additions for this document

- Add a diagram of data file dependencies.
- Add expected wall-time ranges per step on your hardware.
- Add a quick troubleshooting appendix (PySCF memory, CHARMM env, checkpoint naming).